

Comparison in C++

Committee: ISO/IEC JTC1 SC22 WG21 EWG Evolution

Document Number: P0100r2

Title: Comparison in C++

Date: 2016-11-27

Authors: Lawrence Crowl

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Audience: EWG Evolution

Abstract

We explain a significant insecurity in the way programmers use comparison in C++. We develop facilities for making comparisons sound and efficient, primarily by separating computational domain comparisons from utility algorithm comparisons. In the process, we correct an efficiency problem.

Contents

[Introduction](#)

[Definition of Equivalence and Order](#)

[Algorithm Order Requirements](#)

[Correctness](#)

[Analysis of Floating Point](#)

[Not-a-Number](#)

[Signed Zero](#)

[Other Scalar Types](#)

[User-Defined Types](#)

[Algorithms for Comparing Composite Types](#)

[Memory Representation](#)

[Unanimous](#)

[Uncontended](#)

[Lexicographical](#)

[Order Summary](#)

[Domain Computation versus Utility Algorithms](#)

[Proposal Development](#)

[Boolean Comparison Functions](#)

[Consistency Between Relations](#)

[Performance](#)

[Checking](#)

[Function Defaults](#)

[Generic Less and Equal](#)

[Generic Order](#)

[Explicit Defaults for Order](#)

[Operator Defaults](#)

[Outline of Wording Changes](#)

[Revisions](#)

[References](#)

Introduction

Comparisons have many different uses: partitioning over the domain of the type, sorting, memoization, and iteration. As a consequence, comparison is heavily used. As much as 50% of all instructions are comparisons and their subsequent branches. Even extremely low error rates will result in common program failures. These different uses have different requirements. We show that the current language fails to completely address those differences, which leads to potential failure. We also show how to address those differences.

Definition of Equivalence and Order

A comparison is a query on a relation that provides equivalence, order, or both.

All equivalence and order relations are transitive ($a \sim b$ and $b \sim c$ implies $a \sim c$)

All equivalence relations are reflexive ($a \sim a$) and symmetric ($a \sim b$ implies $b \sim a$). An equality (or congruence) relation is an equivalence relation that is also substitutable ($a = b$ implies $f(a) = f(b)$).

Strict orders are irreflexive (not $a < a$), i.e. an element is not less than itself. C++ does not use non-strict orders, so the strict adjective is often implied in the remainder of this paper. Strict orders are also asymmetric ($a < b$ implies not $b < a$) These axioms together constitute a partial order. A weak order also partitions the elements into ordered equivalence classes, that is being unordered with respect to an element is a transitive relation ($a < b$ implies $a < c$ or $c < b$). A total order is trichotomous (exactly one of $a < b$, $a = b$, $a > b$) where equality is defined above.

category	properties	relation	properties
equivalence	reflexive: $a \sim a$ symmetric: $a \sim b$ implies $b \sim a$ transitive: $a \sim b$ and $b \sim c$ implies $a \sim c$	(weak) equivalence	
		equality (congruence)	substitutable: $a = b$ implies $f(a) = f(b)$
order	irreflexive: not $a < a$ asymmetric: $a < b$ implies not $b < a$ transitive: $a < b$ and $b < c$ implies $a < c$	(strict) partial order	
		(strict) weak order	ordered partitions: $a < b$ implies $a < c$ or $c < b$
		(strict) total order	trichotomous: exactly one of $a < b$, $a = b$, $b < a$

Algorithm Order Requirements

The algorithms have requirements on the strength of the order they require.

algorithm	requirements
partitioning the domain for computation	specific to the type
topological sort	partial order
inconsistent normal sort	weak order
consistent normal sort	total order
indexing	total order
memoization	weak order and equality

Correctness

The C++ standard, by default, uses the comparison operators as though they implemented the requirements above. Programmers may not often understand the requirements or even know if the types they are using meet those requirements. The consequences of failing to meet them can be significant, for example: sorting does not sort, only shuffles data around; memoization yields wrong results; and memoization fills up with unused buckets.

Analysis of Floating Point

IEEE floating-point arithmetic has several kinds of special values: positive and negative zero, positive and negative infinity (aka Inf), positive and negative not-a-number (aka NaN). The definition for comparison between them implies a strength in the comparison relations. See section "5.11 Details of comparisons predicates" of [IEEE](#).

Not-a-Number

The expression `NaN==NaN` is false, so the operation is not reflexive. So, the floating-point `operator==` does not form an equivalence relation. When NaN values are part of a sequence of memoization calls, they never compare equal to prior queries, and so every query not only fails to use prior computation, it allocates a new memo for every call. That is, NaNs induce pathological behavior into memoization.

For a normal floating values $x < y$, neither $x < \text{NaN}$ nor $\text{NaN} < y$, and therefore floating-point does not have ordered partions. So, the floating-point `operator<` does not form weak order and therefore does not form a total order. It is, however, a partial order.

A partial order is insufficient for good behavior with `std::sort`. With g++ 4.8.2, the input `{-NaN, -1.0, +0.0, +1.0, +NaN}` in various permutations will yield wildly different output:

```
{ -NaN, -1.0, +0.0, +1.0, +NaN }
{ +NaN, -1.0, +0.0, +1.0, -NaN }
{ -1.0, -NaN, +0.0, +NaN, +1.0 }
```

```
{-1.0, +0.0, +1.0, -NaN, +NaN}
{-1.0, -NaN, +1.0, +NaN, +0.0}
{+NaN, -NaN, -1.0, +0.0, +1.0}
```

The problem here is that an incomparable value tends to partition the sort.

The advice to "do not use NaN" may lead to rare and subtle failures, particularly because comparisons on NaN do not produce a signal. The consequences within large software, with often weak, if any, numerical analysis are mostly unknowable.

Signed Zero

Even in the absence of NaN, signed zero still causes problems. While `+0==0` is `true`, `1.0/-0.0` is `-Inf` and `1.0/+0.0` is `+Inf`. (Other functions, e.g. `atan2` and `log`, behave differently with `-0.0` and `+0.0`. That is `operator==` fails substitutability and therefore, even in the absence of NaN, floating-point `operator==` not an equality comparison, though it is an equivalence.

Because `operator==` is not an equality, memoization yields incorrect results. The first query on an element in an equivalence class is taken as the result of all queries on that class. For example, memoing the function `1/x` with the sequence `{+0.0, -0.0, -0.0}` yields `{+Inf, +Inf, +Inf}` rather than `{+Inf, -Inf, -Inf}`

In the absence of NaN, `operator<` provides a weak order. This meets the requirements of `std::sort`, but may surprise programmers because the order sort output depends on the order of the sort input. Given `{-1.0, -0.0, +0.0, -0.0, +1.0}` in various permutations will yield different output:

```
{-1.0, -0.0, +0.0, -0.0, +1.0}
{-1.0, -0.0, +0.0, -0.0, +1.0}
{-1.0, -0.0, -0.0, +0.0, +1.0}
{-1.0, +0.0, -0.0, -0.0, +1.0}
```

Signed zeros result from operations that underflow, and since inexact signals are rarely enabled, one cannot readily dismiss signed zero as too rare to consider. The consequences within large software, with often weak, if any, numerical analysis are mostly unknowable.

Other Scalar Types

Most current systems use a flat linear memory space with simple pointers, which are simply placed in a total order. But flat memory spaces were not always the case and may not be so in the future. While the operating system may be able to provide a total order, it may not do so efficiently. Comparison between pointers to within the same object is almost certain to be efficient. Comparison between general pointers may require an operating system call to translate a capability to a segment index, and is therefore unlikely to be efficient. Distinguishing between such comparisons has the best chance of preserving C++ program performance on future systems.

Most modern systems use two's complement arithmetic for signed numbers. However, the standard also supports one's complement and signed magnitude representations. Both of these formats have signed zero and thus have some of the same issues as IEEE floating-point numbers.

User-Defined Types

More importantly, user-defined types are often composite types. Programmers write the comparison operators to partition the value domain in ways that serve algorithms using those types. It is easy to produce a partially ordered type in this environment, and so those operators are increasingly unlikely to meet the algorithmic requirements. Some types explicitly do not provide comparison operators simply to prevent inappropriate comparison, e.g. `std::complex` provides equality operators but not relational operators.

Algorithms for Comparing Composite Types

There are several general algorithms for comparing composite types.

Memory Representation

Using `memcmp` provides a total order based on memory representation. Unfortunately, it will likely not deliver results consistent with operators. More importantly, it will certainly not deliver expected results for types with indirect data, such as `std::string`. Due to these issues, we will not consider it further.

Unanimous

The algorithm requires that all corresponding fields must be less:

$$T < U \text{ iff for all } i, T[i] < U[i]$$

The composite relation is obviously irreflexive and transitive, so it provides a partial order. But while $(2,2) < (3,3)$, neither $(2,2) < (1,4)$ nor $(1,4) < (3,3)$, so resulting order is neither weak nor total.

Uncontended

The algorithm requires that at least one field must be less, and none be greater:

$$T < U \text{ iff (exists } i \text{ such that } T[i] < U[i]) \text{ and (forall } j \neq i, \text{ not } U[j] < T[j])$$

The composite relation is obviously irreflexive. Given a field with a partial order, though, the relation is not transitive, e.g. $(2,1) < (\text{NaN},2)$ and $(\text{NaN},2) < (1,3)$ but not $(2,1) < (1,3)$. So, the resulting relation is not even an order. A partial order breaks the algorithm.

Even with fields totally ordered, $(2,2) < (3,3)$ but neither $(2,2) < (1,4)$ nor $(1,4) < (3,3)$, so the resulting relations are neither weak nor totally ordered. They are, however, partially ordered.

Lexicographical

The algorithm requires that all fields compare equal before the first field that compares less:

$$T < U \text{ iff exists } i \text{ such that } (T[i] < U[i] \text{ and (forall } j < i, \text{ not } U[j] < T[j]))$$

The composite relation is obviously irreflexive. Given a field with a partial order, though, the relation is not transitive, e.g. $(2,1) < (\text{NaN},2)$ and $(\text{NaN},2) < (1,3)$ but not $(2,1) < (1,3)$. So, the resulting relation is not even an order. A partial order breaks the algorithm.

When fields are at least weakly ordered, the order is defined by the first non-equivalent field, which in turn is either weakly or totally ordered. So, the stength of the composite relation is weak if any field is weak, otherwise it is total.

Order Summary

The order produced by the algorithms is summarized as follows.

	resulting order for weakest field order being		
algorithm	partial	weak	total
unanimous	partial	partial	partial
uncontended	failure	partial	partial
lexicographical	failure	weak	total

Where the algorithms do not fail, lexicographical refines uncontended which refines unanimous.

Domain Computation versus Utility Algorithms

The core of the correctness problem is in using the same operators for both domain-specific computation and utility algorithms. The solution is separate those two uses.

The floating-point standard [\[IEEE\]](#) expects NaNs to be compared. It has gone out of its way to define the results of such comparisons, presumably to support robust numeric computations. Likewise division is a well-define operation that both admits NaN as an argument and may generate NaN as a result. The IEEE standard recognizes that comparisons with NaN are a problem for utility algorithms. To solve this problem, [\[IEEE\]](#) provides the `totalOrder` operation for use with those algorithms.

Because we cannot change the existing operator meanings, we must leave operators to user-defined semantics for their domain of computation and introduce new functions for utility algorithms.

To be clear, **robust standard algorithms should never use comparison operators**. They should use comparator functions that provide the semantics that they require.

Proposal Development

Standard algorithms generally take a comparator function as a (often default) template argument. These algorithms generally do

not need a concise notation for comparison. So, we add new functions for comparison.

Boolean Comparison Functions

We introduce the following functions in namespace `std` in analogy to `std::less`:

```
template<typename T> bool partial_less(const T&,const T&);
template<typename T> bool weak_less(const T&,const T&);
template<typename T> bool total_less(const T&,const T&);
```

Where these functions provide a partial order, a weak order, and a total order, respectively.

Separate facilities for a total order beyond the operators is not new. IEEE floating-point [\[IEEE\]](#) provides a `totalOrder` function in section 5.10 Details of totalOrder predicate. Likewise Java provides both `Comparable` and `Comparator` which provide a total order [\[Java\]](#).

We also provide the following complementary functions, in analogy to `std::equal_to`:

```
template<typename T> bool partial_unordered(const T&,const T&);
template<typename T> bool weak_equivalence(const T&,const T&);
template<typename T> bool total_equal(const T&,const T&);
```

Where these functions return `true` when neither argument is less than the other using the corresponding functions. The function `weak_equivalence` forms an equivalence and the function `total_equal` forms an equality.

Specializations for these functions will need to be provided for all the standard types. Fortunately, they are a pure extension.

Consistency Between Relations

Programmers will likely expect the above functions to deliver consistent results. We may not be able to provide that consistency in the standard, but we can define it.

A function g *refines* a function f when for every a and b , $f(a,b)$ implies $g(a,b)$. That is, if a call to f is true, so is the corresponding call to g .

A set of order functions is consistent when

- `total_less(a,b)` refines `weak_less(a,b)`
- `weak_less(a,b)` refines `partial_less(a,b)`
- `weak_equivalence(a,b)` refines `total_equal(a,b)`
- `partial_unordered(a,b)` refines `weak_equivalence(a,b)`
- `total_equal(a,b)` equals `!total_less(a,b) && !total_less(b,a)`
- `weak_equivalence(a,b)` equals `!weak_less(a,b) && !weak_less(b,a)`
- `partial_unordered(a,b)` equals `!partial_less(a,b) && !partial_less(b,a)`

A set of order functions is consistent with an operator set when

- `total_less(a,b)` refines `a<b`
- `total_less(b,a)` refines `a>b`
- `a==b` refines `total_equal(b,a)`
- `weak_less(a,b)` refines `a<b` or vice-versa
- `weak_less(b,a)` refines `a>b` or vice-versa
- `a==b` refines `total_equivalence(b,a)` or vice-versa
- `partial_less(a,b)` refines `a<b` or vice-versa
- `partial_less(b,a)` refines `a>b` or vice-versa

The 'vice-versa' above is present because the operators may be either weaker or stronger than the comparators. The `partial_unordered` rule is not present by design, as it seems inconsistent with the operator in intent.

Performance

The standard typically uses a lexicographical comparison, which is built on `operator<`. The algorithms often require that they visit a field twice, i.e. call both $a.f < b.f$ and $b.f < a.f$. For a composite type with l layers and n fields per layer, the worst case complexity is $O((2*n)^{(l+1)})$.

Algorithms that visit each field pair once can be substantially cheaper.

Rather than rely on a boolean less, we adopt the approach of `strcmp` et.al. and return a three-state value indicating less, greater, or neither. In addition to historical C funtions, the approach is also used by Java `Comparable` and `Comparator` [\[Java\]](#).

The functions are `partial_order`, `weak_order`, and `total_order`.

Checking

At present, there is generally no diagnostic when an operator< does not meet the requirements of the algorithm that uses it.

We create different function signatures for the different order functions by having different three-state return types. Algorithms embodying certain requirements would require a comparison function that returned the appropriate type.

The three-state return types are:

```
enum class partial_ordering { less, unordered, greater };
enum class weak_ordering { less, equivalent, greater };
enum class total_ordering { less, equal, greater };
```

and the corresponding order functions are

```
template<typename T> partial_ordering partial_order(T,T)
template<typename T> weak_ordering weak_order(T,T)
template<typename T> total_ordering total_order(T,T)
```

Function Defaults

There are a potentially significant number of new specializations introduce by the above functions. We can reduce some of the coding with generic implementations or explicit defaults.

Generic Less and Equal

Given order functions, we can infer the less and equal functions.

```
template<typename T> bool total_less(T a, T b) {
    return total_order(a,b) == total_ordering::less;
}
template<typename T> bool total_equal(T a, T b) {
    return total_order(a,b) == total_ordering::equal;
}
```

Similar definitions exist for the other strengths.

These functions can of course be specialized for efficiency.

Generic Order

A total order is also a weak order and a weak order is also a partial order, so we can use a generic implementation for a weaker version using a stronger version.

```
template<typename T> weak_ordering weak_order(T a, T b) {
    total_ordering o = total_order(a,b);
    switch ( o ) {
        case total_ordering::less: return weak_ordering::less;
        case total_ordering::greater: return weak_ordering::greater;
        case total_ordering::equal: return weak_ordering::equivalent;
    }
}
```

A similar definition exists for `partial_order`. However, there is no generic version for `total_order`.

These functions can of course be specialized for efficiency.

Explicit Defaults for Order

For classes of one field, the defaults for order functions can simply reflect the order of that field.

We can provide an explicit default definitions for a composite total order using the lexicographical algorithm. Such a default would require that all fields also provide a total order. However, the lexicographical algorithm on fields may not be consistent

with the operators. Without automatic consistency, it is not clear that we should provide default definitions, and we should certainly not provide implicit defaults.

Much the same argument applies to a default for composite weak order, except that the requirement is that all fields provide at least a weak order.

There are two reasonable algorithms for a composite partial order. One provides more refined results while the other has more generality. A default here seems less than obvious.

Operator Defaults

Given the approach of separating comparison for utility algorithms from domain-specific comparison, operator defaults should support the programmer, not enforce a goal of total order. However, many programmers may desire a goal of total order, and defaults can help them get there.

Whether or not the default is explicit is very important. An incorrect implicit default must be corrected with either `=delete` or a new function body. This task is a cognitively hard because how do you know to correct what you did not write? An incorrect explicit default is less likely to be a problem because the programmer must actively invoke the default, which means the programmer does at least some thinking. So, the criteria for an implicit default should be "doing something other than the default is so rare that it is better to introduce a bug into a program than to make programmers write explicit defaults".

For types of a single field, the default should be to simply reflect the operator for the field. This approach is consistent with wrapper abstractions.

For classes that have more than one field, or for those that have one field but no corresponding operator, there are some uncontroversial defaults. These could easily be *implicit* defaults.

1. Derive $a \neq b$ from $!(a == b)$.
2. Derive $a > b$ from $b < a$.

There are also some more controversial defaults.

1. Derive $a \leq b$ from $a < b \mid a == b$ versus from $!(a > b)$.
2. Derive $a \geq b$ from $a > b \mid a == b$ versus from $!(a < b)$.

The former directly reflects the reading of the operator names while the later assumes at least a consistent weak ordering. The latter is not true of floating point, and probably many user-defined types. As a consequence, a default for `<=` of less-or-equal is reasonable as an *implicit* default, but a default of not-greater is only reasonable as an *explicit* default.

The remaining `operator<` and `operator==` are too specific to the value domain and the functions that operate on it to have reasonable *implicit* defaults when the structure has more than one field. For `explicit` defaults, using the strongest available comparator function seems reasonable. Note that when these operations are defaulted with three-way comparators, the implicit defaults suggested above for the `<=` and `>=` operators become single-pass algorithms via the as-if rule.

Outline of Wording Changes

The first stage of work is add comparators to all standard types. This process may take some time as template type arguments will not initially have comparators to build from.

- Define total comparators for integer types. The other orders are equivalent.
- Define partial, weak and total comparators for floating types. They are distinct.
- Define total comparators for integer types. The other orders are equivalent.
- Define total comparators for `type_info` that refine `operator==` and `before`.
- Define total comparators for `error_code` that refine operators.
- Define total comparators for `error_category` that refine operators.
- Define total comparators for `unique_ptr` that refine operators.
- Define total comparators for `shared_ptr` that refine operators.
- Define `total_equal` for `bitset` that refines `operator==`.
- Define `total_equal` for `allocator` that refines `operator==`.
- And others.

Make versions of algorithms that take trinary comparators.

Cleanup existing operator definitions. E.g. change definitions of 20.2.1 operators `<=` and `>=`.

Add explicit defaults for user-defined operators >, !=, >= and <=.

Transition algorithms to use trinary comparators by default.

Revisions

This paper revises [P0100R1 Comparison in C++](#).

- Add abstract.
- Make "Algorithms for Comparing Composite Types" a top-level section.
- Correct "Memory Representation" section heading.
- Separate out domain-specific section from correctness.
- Note that [\[IEEE\]](#) separately provides for utility algorithms.
- Gather proposal development into a single section.
- Move consistency between equal and less into bullet points.
- Add a paragraph on choosing implicit versus explicit defaults.
- Clarify implicit versus explicit defaults for operators.
- Add note on single-pass operators.
- Minor spelling and grammar corrections.

P0100R1 revised [P0100R0 Comparison in C++](#).

- Add missing text in the document.

[P0100R0](#) revises [N4367 Comparison in C++](#).

- Add comment on compare frequency.
- Split normal sort into inconsistent and consistent versions.
- Change weak_equal to weak_equivalence.
- Add section on outline of wording changes.

References

[IEEE]

IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2008, <http://ieeexplore.ieee.org/xpl/mostRecentIssue.jsp?punumber=4610933>

[Java]

The Java™ Tutorials, Object Ordering, <http://docs.oracle.com/javase/tutorial/collections/interfaces/order.html>